

ML MATH PREREQUISITES

Bài Tập Tự Học & Luyện Tập

Linear Algebra • Calculus • Probability & Statistics

Machine Learning in Data Sciences

🔗 Hướng dẫn: Điền câu trả lời vào ô trống dưới mỗi câu hỏi. Các bài có ký hiệu 📄 kèm code NumPy/Python. Câu hỏi 💡 là câu hỏi tư duy — ghi câu trả lời bằng lời.

Mục tiêu học tập

Phần	Sau khi hoàn thành, bạn sẽ...
📐 Linear Algebra (Slide 5–17)	Thao tác vector & ma trận, hiểu dot product, norm, inverse
📈 Calculus (Slide 21–25)	Tính đạo hàm, partial derivative, gradient, áp dụng chain rule
🎲 Probability & Stats (Slide 28–33)	Bayes theorem, distributions, expectation, covariance
🏆 Bài tổng hợp	Tự xây Logistic Regression từ toán thuần túy — không dùng sklearn

PHẦN 1 — LINEAR ALGEBRA

Slide 5 Scalars & Vectors

Bài 1.1 — Scalars và Vectors

“ Vector: ordered list of numerical values. Each data point is expressed as a vector of features. e.g. a house: [size, rooms, age] = [120, 3, 10] $\in \mathbb{R}^3$ ”

Bối cảnh: Trong ML, mỗi quan sát (nhà, email, sinh viên...) được biểu diễn là một vector.

Câu A — Tạo 3 vector căn nhà với 4 đặc trưng: [diện_tích_m², số_phòng, tuổi_nhà, số_tầng]

```
import numpy as np
house_A = np.array([__]) # chung cư nhỏ: 85m², 2 phòng, 5 tuổi, tầng 3
house_B = np.array([__]) # chung cư vừa: 120m², 3 phòng, 10 tuổi, tầng 7
house_C = np.array([__]) # nhà mặt đất: 200m², 4 phòng, 2 tuổi, tầng 1
print(house_A.shape) # phải là (4,)
```

 **Kết quả khi in house_A.shape:** — (4,) — vì vector có 4 chiều

```
import numpy as np
house_A = np.array([85, 2, 5, 3])
house_B = np.array([120, 3, 10, 7])
house_C = np.array([200, 4, 2, 1])
print(house_A.shape)
```

Câu B — Scalar: tại sao learning rate $\alpha = 0.01$ là scalar, không phải vector?

 **Giải thích:** — α là một số đơn, không có chiều (dimensionless)

Learning rate alpha = 0.01 là scalar vì nó là một số đơn, không có chiều
=> Chỉ biểu thị độ lớn (magnitude)

💬 Tư duy: Nếu thêm đặc trưng 'khoảng cách tới trung tâm (km)', vector nhà thuộc $\mathbb{R}^?$

 **Câu trả lời:**

Nếu thêm một đặc trưng nữa, vector nhà sẽ có 5 chiều.

Slide 6 Vector Addition

Bài 1.2 — Vector Addition

“ $u + v = [u_1+v_1, u_2+v_2, \dots, u_n+v_n]$ — Cộng từng phần tử tương ứng

Câu A — Tính $u + v$:

```
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])
uv_sum = ____ # điền phép cộng
# Kết quả mong đợi: [5, 7, 9]
```

 **Kết quả uv_sum:** — [5, 7, 9]

```
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])
uv_sum = u + v
```

Câu B — ML Application: Simple Ensemble (trung bình prediction 2 model):

```
pred_model1 = np.array([0.2, 0.7, 0.5, 0.9])
pred_model2 = np.array([0.3, 0.6, 0.4, 0.8])
ensemble_avg = ____ # trung bình cộng 2 vector
# Model đồng thuận mạnh ở vị trí nào?
```

 **Kết quả ensemble_avg:**

```
ensemble_avg = np.array([0.25, 0.65, 0.45, 0.855])
```

Model đồng thuận mạnh ở vị trí: Index 0 (0.2 và 0.3) và Index 2 (0.5 và 0.4).

  **Tại sao ensemble thường cho kết quả tốt hơn 1 model đơn lẻ?**

Vì nó giúp trung bình hóa các sai số (variance) của các mô hình đơn lẻ, đem lại dự đoán ổn định và đáng tin cậy hơn.

Slide 7 Scalar Multiplication

Bài 1.3 — Scalar Multiplication & Gradient Descent Step

“ $c \cdot u = [c \cdot u_1, c \cdot u_2, \dots, c \cdot u_n]$

Câu A — Tính $2 \times [1, 2, 3]$:

```
u = np.array([1, 2, 3]); c = 2
scaled = ____
# Kết quả: [2, 4, 6]
```

 **Kết quả:** — [2, 4, 6]

scaled = [2, 4, 6]

Câu B — Gradient Descent update: $W_{\text{new}} = W_{\text{old}} - \alpha \times \text{gradient}$

```
W_old = np.array([0.5, -0.3, 0.8])
gradient = np.array([0.1, -0.2, 0.05])
alpha = 0.01
W_new = ____ # W_old - alpha * gradient
```

Kết quả W_new:

W_new = [0.499, 0.298, 0.7995]

💬 Tại sao ta TRỪ gradient (không cộng) khi cập nhật weights?

Giải thích:

Vì gradient chỉ ra hướng "tăng dốc nhất" của hàm loss. Ta muốn tìm điểm cực tiểu (giảm loss), nên phải bước đi ngược hướng với đạo hàm (tức là làm phép trừ).

Slide 8 Dot Product

Bài 1.4 — Dot Product: Trái tim của Linear Models

“ $u \cdot v = \sum_i u_i v_i$ — Nhân từng cặp phần tử rồi cộng lại

Câu A — Tính $u \cdot v$:

```
u = np.array([1, 2, 3])
v = np.array([4, 5, 6])
dot_uv = np.dot(u, v) # hoặc u @ v
# Tính thủ công:  $1 \times 4 + 2 \times 5 + 3 \times 6 = ?$ 
```

Tính thủ công (điền số): — $1 \times 4 + 2 \times 5 + 3 \times 6 =$ ____

4 + 10 + 18 = 32

Kết quả dot_uv: — 32

dot_uv = 32

Câu B — Linear Prediction: $y = w \cdot x + b$ (mô hình dự đoán xác suất nhận học bổng)

```
x = np.array([320, 8.5, 1.0])      # [GRE, CGPA, Research]
w = np.array([0.002, 0.05, 0.03]) # trọng số model
b = -0.5                          # bias
y_pred = _____               #  $w \cdot x + b$ 
```

 **Kết quả y_{pred} (xác suất được nhận):**

$320 \times 0.002 + 8.5 \times 0.05 + 1.0 \times 0.03 - 0.5 = 0.595$

 Slide 9–10 Vector Norm & Cosine Similarity

Bài 1.5 — Vector Norms & Regularization

“ L2 norm: $\|u\|_2 = \sqrt{\sum_i u_i^2}$ | L1 norm: $\|u\|_1 = \sum_i |u_i|$

Câu A — Tính L1 và L2 norm của $u = [1, 2, 3, 4, 5]$:

```
u = np.array([1, 2, 3, 4, 5])
l2_norm = _____ # np.linalg.norm(u)
l1_norm = _____ # np.linalg.norm(u, ord=1)
```

 **L2 norm = $\sqrt{(1+4+9+16+25)} = \sqrt{55} \approx$ — 7.4162**

7.416

 **L1 norm = $1+2+3+4+5 =$ — 15**

15

Câu B — L2 & L1 Regularization penalties ($\lambda = 0.1$):

```
weights = np.array([0.8, -0.3, 0.05, 1.2, -0.9])
l2_penalty = 0.1 * np.linalg.norm(weights)**2 # Ridge
l1_penalty = 0.1 * np.linalg.norm(weights, ord=1) # Lasso
```

 **L2 penalty (Ridge) =**

`l2_penalty = 29825`

 **L1 penalty (Lasso) =**

`l1_penalty = 0,325`

💬 Tại sao Lasso (L1) tạo ra weights = 0 trong khi Ridge (L2) không?

 **Giải thích:**

`0.325`

Câu C — Cosine Similarity giữa $a=[1,2,3]$ và $b=[4,5,6]$:

```
cosine_sim = np.dot(a,b) / (np.linalg.norm(a) * np.linalg.norm(b))
```

 **Cosine similarity (kết quả, khoảng 0–1):**

`cosine_sim = 0.974`

Slide 11–15 Matrices & Operations

Bài 1.6 — Matrix Multiplication & Transpose

“ $C = XY \in \mathbb{R}^{m \times p}$, $C_{ij} = \sum_k X_{ik} \times Y_{kj}$ | $(X^T)_{ij} = X_{ji}$ ”

Data matrix X — 4 sinh viên, 3 đặc trưng: [điểm_toán, GPA, số_dự_án]

```
X = np.array([[85, 8.2, 2], [92, 9.0, 3], [70, 7.5, 1], [88, 8.7, 4]]) # (4,3)
W = np.array([[0.4, 0.1], [0.3, 0.5], [0.2, 0.3]]) # (3,2)
```

Câu A — Trước khi tính, đoán shape của $X @ W$:

 **Shape của $X @ W = (\underline{\quad}, \underline{\quad})$ — Gợi ý: $(m,k) @ (k,n) = (m,n)$**

`$X @ W = (4, 2)$`

Câu B — Tính $Z = X @ W$ và $X.T$:

```
Z = ____ # X @ W
X.T = ____ # X.T
```

 **Shape thực tế của Z:** — (4, 2)

$Z = (4, 2)$

 **Shape của X.T:** — (3, 4)

$X.T = (3, 4)$

💬 Trong neural network, $X @ W$ tương đương thao tác gì?

 **Giải thích:**

Tương đương thao tác - Tính tổng có trọng số (weighted sum) của toàn bộ các đặc trưng cho tất cả dữ liệu đầu vào (cùng lúc) qua một layer.

 Slide 14–17 Hadamard, Identity, Inverse, Norms

Bài 1.7 — Hadamard, Inverse & Matrix Norms

Câu A — Hadamard (element-wise $\neq$ matrix multiplication):

```
A = np.array([[1,2,3],[4,5,6]])
B = np.array([[7,8,9],[1,2,3]])
hadamard = ____ # A * B (element-wise)
mat_mult = ____ # A @ B.T (matrix mult - cần transpose vì shape không khớp)
```

 **Hadamard $A \odot B$ — hàng 1:** — $[[7,16,27],...]$

$[[7, 16, 27], ...]$

Câu B — Kiểm tra tính chất nghịch đảo: $M^{-1} \cdot M = I$:

```
M = np.array([[2.0, 1.0], [5.0, 3.0]])
M_inv = np.linalg.inv(M)
print(np.round(M_inv @ M, 8)) # phải ra ma trận đơn vị
```

 **M_inv @ M (kết quả, làm tròn):**

[[1. 0.], [0. 1.]]

Câu C — Matrix Norms (L1 và L2/Frobenius):

```
X_mat = np.array([[1,2,3], [4,5,6], [7,8,9]])
l1_norm = np.sum(np.abs(X_mat)) # tổng giá trị tuyệt đối
l2_norm = np.sqrt(np.sum(X_mat**2)) # Frobenius norm
```

 **L1 norm = tổng tất cả phần tử =** — 45

45

 **L2 (Frobenius) norm ≈**

16.88

PHẦN 2 — CALCULUS

Slide 21 Derivative

Bài 2.1 — Derivative & MSE Loss

“ $f'(x)$: rate of change. Power rule: $d/dx(x^n) = n \cdot x^{n-1}$. Sum rule: $(f+g)' = f' + g'$ ”

Câu A — Tính đạo hàm (điền vào ô trống):

Hàm $f(x)$	Đạo hàm $f'(x)$
$f(x) = x^2$	$f'(x) = \underline{\quad}$ (tại $x=3$: $f'(3) = \underline{\quad}$)
$f(x) = 3x + 5$	$f'(x) = \underline{3}$
$f(x) = x^3 + 2x^2 - x$	$f'(x) = \underline{3x^2 + 4x - 1}$
$f(x) = c$ (hằng số)	$f'(x) = \underline{0}$

Câu B — Gradient của MSE Loss: $L(w) = (y - w \cdot x)^2$

Áp dụng chain rule để tính dL/dw :

```
# Tại w=0.5, x=2.0, y=1.0
# dL/dw = -2 * x * (y - w*x)
# Bước 1: tính prediction  $\hat{y} = w \cdot x = \underline{1.0}$ 
# Bước 2: tính error =  $y - \hat{y} = \underline{0}$ 
# Bước 3:  $dL/dw = -2 * x * \text{error} = \underline{0}$ 
```

 **dL/dw (gradient tại điểm trên):**

💬 Gradient dương $\rightarrow$ ta nên tăng hay giảm w để giảm loss?

 **Giải thích:**

Nếu gradient dương, ta nên giảm w (vì đi ngược hướng đạo hàm) để giảm loss.

Slide 22 Partial Derivatives

Bài 2.2 — Partial Derivatives

“ $f(x,y) = x^2 + y^2 \rightarrow \partial f/\partial x = 2x$ (giữ y cố định), $\partial f/\partial y = 2y$ ”

Câu A — Tính partial derivatives tại (x=2, y=3):

```
# f(x, y) = x^2 + y^2
# ∂f/∂x = ____ (tại x=2) = 4
# ∂f/∂y = ____ (tại y=3) = 6
```

 **∂f/∂x tại x=2:** — $2x = 2 \times 2 = 4$

 **∂f/∂y tại y=3:** — $2y = 2 \times 3 = 6$

$2 \times 3 = 6$

Câu B — MSE với 2 tham số: $L(w_1, w_2) = (y - w_1x_1 - w_2x_2)^2$

Tính $\partial L / \partial w_1$ và $\partial L / \partial w_2$ tại: $y=1.0$, $x_1=2.0$, $x_2=3.0$, $w_1=0.3$, $w_2=0.2$

```
# ŷ = w1*x1 + w2*x2 = 1.2
# error = y - ŷ = -0.2
# ∂L/∂w1 = -2 * x1 * error = 0.8
# ∂L/∂w2 = -2 * x2 * error = 1.2
```

 **Các bước tính và kết quả $\partial L / \partial w_1$, $\partial L / \partial w_2$:**

💬 Tại sao $\partial L / \partial w_2$ lớn hơn $\partial L / \partial w_1$ trong ví dụ này?

 **Giải thích:**

$\partial L / \partial w_2$ lớn hơn vì đầu vào x_2 (3.0) lớn hơn x_1 (2.0), sự thay đổi của w_2 sẽ phóng đại mức độ ảnh hưởng lên loss nhiều hơn

Bài 2.3 — Gradient & Gradient Descent

“ ∇f = vector of all partial derivatives. Update rule: $W = W - \alpha \cdot \partial L / \partial W$ ”

Câu A — Gradient của $f(x, y) = x^2 + y^2$ tại điểm (x=1, y=2):

```
# ∇f = [∂f/∂x, ∂f/∂y] = [2x, 2y]
# Tại (1, 2): ∇f = [__, __]
```

 **∇f tại (1, 2):** — [2, 4]

[2, 4]

Câu B — Chạy 5 bước Gradient Descent (tay hoặc code), bắt đầu từ (3.0, 4.0), $\alpha=0.1$:

Bước	Điểm (x, y)	f(x,y)	Gradient ∇f
0 (start)	(3.0, 4.0)	$9 + 16 = 25$	[6, 8]
1	(2.4, 3.2)	16	[4.8, 6.4]
2	(1.92, 2.56)	10.24	[3.84, 5.12]
3	(1.536, 2.048)	6.5536	[3.072, 4.096]
4	(1.2288, 1.6384)	4.1943	[2.4576, 3.2768]
5	(0.9830, 1.3107)	2.6844	[1.9661, 2.6214]

💬 Điểm (x, y) đang hội tụ về đâu? Tại sao đó là điểm tối ưu?

 **Nhận xét:**

Điểm đang hội tụ dần về (0, 0). Đó là điểm tối ưu vì tại đó $f(x,y) = x^2 + y^2$ đạt giá trị cực tiểu toàn cục là 0.

 Slide 25 **Chain Rule**

Bài 2.4 — Chain Rule & Backpropagation

“ $df/dx = (df/dg) \cdot (dg/dx)$ — Nền tảng của Backpropagation trong Neural Networks

Câu A — Tính đạo hàm tổng hợp tại x=3: f(g(x)), trong đó f(u)=u², g(x)=3x+1:

```
# Bước 1: u = g(3) = 3×3+1 = 10
# Bước 2: df/du = 2u = 20
# Bước 3: du/dx = 3
# Bước 4: df/dx = (df/du) × (du/dx) = 20 × 3 = 60
# Kiểm tra trực tiếp: 6(3x+1) tại x=3 = 6×__ = __
```

 **Kết quả df/dx tại x=3:** — 60

Câu B — Chain rule qua 1 neuron: $x \rightarrow [z=wx+b] \rightarrow [a=\text{ReLU}(z)] \rightarrow [L=(y-a)^2]$

Tính dL/dw tại: $x=2.0$, $w=0.5$, $b=0.1$, $y=1.0$

```
# Forward pass:
#  $z = w \cdot x + b = \underline{1.1}$ 
#  $a = \text{ReLU}(z) = \max(0, z) = \underline{1.1}$ 
#  $L = (y - a)^2 = \underline{0.01}$ 

# Backward pass (chain rule):
#  $dL/da = -2 \cdot (y-a) = \underline{0.2}$ 
#  $da/dz = \text{ReLU}'(z) = 1 \text{ if } z>0 \text{ else } 0 = \underline{1}$ 
#  $dz/dw = x = \underline{2.0}$ 
#  $dL/dw = dL/da \times da/dz \times dz/dw = \underline{0.4}$ 
```



dL/dw (điền kết quả từng bước):

💬 Nếu $z < 0$ ($\text{ReLU} = 0$), điều gì xảy ra với dL/dw ? Đây là vấn đề gì?



Giải thích (gợi ý: 'dying ReLU'):

Nếu $z < 0$, ReLU trả về 0, khi đó đạo hàm $da/dz = 0$. Kéo theo $dL/dw = 0$. Trọng số sẽ vĩnh viễn không được cập nhật nữa (nơ-ron bị "chết").

PHẦN 3 — PROBABILITY & STATISTICS

Slide 28 Descriptive Statistics

Bài 3.1 — Descriptive Statistics

“ Mean: $\mu = (1/n)\sum x_i$ | Variance: $\sigma^2 = (1/n)\sum (x_i - \mu)^2$ | Std: $\sigma = \sqrt{\text{Var}}$ ”

Dữ liệu: Điểm thi ML của 10 sinh viên: [72, 85, 90, 68, 78, 95, 60, 82, 77, 88]

Câu A — Tính thủ công (tay):

```
scores = [72, 85, 90, 68, 78, 95, 60, 82, 77, 88]
# Mean = (72+85+90+68+78+95+60+82+77+88) / 10 = ____ / 10 = ____
# Median = sắp xếp rồi lấy giữa: [60, 68, 72, 77, ____, 82, 85, 88, 90, 95]
#           = (____ + ____) / 2 = ____
```

 **Tổng điểm = 795**

 **Mean (μ) = 79.5**

 **Median = 77.5**

Câu B — Tính bằng NumPy và so sánh:

```
scores = np.array([72, 85, 90, 68, 78, 95, 60, 82, 77, 88])
mean    = np.mean(scores)    # = ____
var     = np.var(scores)     # = ____
std     = np.std(scores)     # = ____
median  = np.median(scores)  # = ____
```

 **Variance (σ^2) = 111.45**

 Std Dev (σ) = 10.56

Câu C — So sánh 2 lớp. Lớp nào "nguy hiểm" hơn cho model? Tại sao?

```
class_A = [70, 75, 72, 78, 73] # đồng đều  
class_B = [50, 95, 60, 90, 55] # phân cực  
# std_A = ___, std_B = ___
```

 std_A = 2.73 std_B = 18.71

  Class nào có variance cao hơn và điều đó ảnh hưởng đến ML như thế nào?

10.56 --> Dữ liệu phân cực lớn có nhiều cao, khiến mô hình khó học mẫu chung và dễ tổng quát hóa sai

Slide 29 Probability Fundamentals

Bài 3.2 — Probability & Conditional Probability

“ $P(A)$ = favourable / total | $P(A|B) = P(A \cap B) / P(B)$ ”

Bảng dữ liệu: 200 sinh viên, phân theo [Giỏi lập trình] × [Có kinh nghiệm]

	Code=1 (giỏi)	Code=0 (chưa)	Tổng
Exp=1 (có KN)	80	30	110
Exp=0 (chưa)	40	50	90
Tổng	120	80	200

Câu A — Xác suất cơ bản:

```
P(Code=1) = 120/200 = 0.6  
P(Exp=1) = 110/200 = 0.55  
P(Code=1 ∩ Exp=1) = 80/200 = 0.4
```

 Các xác suất cơ bản (điền số thập phân):

Câu B — Conditional probability:

$$P(\text{Code}=1 \mid \text{Exp}=1) = P(\text{Code} \cap \text{Exp}) / P(\text{Exp}) = \frac{0.4}{0.55} = \frac{0.727}{1}$$

$$P(\text{Exp}=1 \mid \text{Code}=1) = P(\text{Code} \cap \text{Exp}) / P(\text{Code}) = \frac{0.4}{0.6} = \frac{0.667}{1}$$

 **P(giỏi code | có kinh nghiệm) = 0.727**

 **P(có kinh nghiệm | giỏi code) = 0.667**

Câu C — Kiểm tra independence. Nếu độc lập: $P(A \cap B) = P(A) \times P(B)$:

 **$P(\text{Code}) \times P(\text{Exp}) = 0.33$ $P(\text{Code} \cap \text{Exp})$ thực tế = 0.4 Độc lập? (Y/N): N**

💬 Giải lập trình và kinh nghiệm có độc lập không? Điều này có nghĩa gì?

 **Giải thích:**

Kỹ năng code giỏi và việc có kinh nghiệm thường đi liền với nhau, tương tác với nhau, chứ không sinh ra một cách ngẫu nhiên và rời rạc.

Bài 3.3 — Bayes' Theorem: Naive Bayes Classifier

“ $P(A|B) = P(B|A) \times P(A) / P(B)$ — Từ prior đến posterior

Bài toán: Phân loại review phim Positive/Negative dựa trên từ xuất hiện

Thông tin	Giá trị
P(Positive) — Prior	0.6
P(Negative) — Prior	0.4
P("tuyệt" Positive)	0.7
P("tuyệt" Negative)	0.1

Câu A — Tính P("tuyệt") bằng law of total probability:

```
# P("tuyệt") = P("tuyệt" | Pos) × P(Pos) + P("tuyệt" | Neg) × P(Neg)
# = 0.7 × 0.6 + 0.1 × 0.4
# = 0.42 + 0.04 = 0.46
```

 P("tuyệt") = 0.46

Câu B — Tính Posterior bằng Bayes:

```
# P(Pos | "tuyệt") = P("tuyệt" | Pos) × P(Pos) / P("tuyệt")
# = ____ × ____ / ____ = ____
# P(Neg | "tuyệt") = P("tuyệt" | Neg) × P(Neg) / P("tuyệt")
# = ____ × ____ / ____ = ____
# Phân loại: ____ (P(Pos | "tuyệt") > P(Neg | "tuyệt"))?
```

 P(Positive | thấy từ 'tuyệt') = 0.913

 Kết quả phân loại: POSITIVE hay NEGATIVE?

POSITIVE

Câu C (Nâng cao) — Naive Bayes với 2 từ 'tuyệt' VÀ 'hay':

P("hay" | Pos) = 0.65, P("hay" | Neg) = 0.15. Giả sử 2 từ độc lập (Naive Bayes):

```
# score_pos ∝ P('tuyệt' | Pos) × P('hay' | Pos) × P(Pos) = 0.273
# score_neg ∝ P('tuyệt' | Neg) × P('hay' | Neg) × P(Neg) = 0.006
# P(Pos | 2 từ) = score_pos / (score_pos + score_neg) = 0.978
```

 P(Positive | 'tuyệt' + 'hay') = 0.978



Tại sao gọi là 'Naive' Bayes? Giả định nào được đưa ra?

Vì nó đưa ra giả định ngây thơ rằng các từ (đặc trưng) xuất hiện hoàn toàn độc lập với nhau (từ "tuyệt" không liên quan gì đến từ "hay").



Slide 31 Probability Distributions

Bài 3.4 — Phân Phối Xác Suất

“ Normal: $f(x) = (1/\sigma\sqrt{2\pi}) \times \exp(-(x-\mu)^2/2\sigma^2)$ — 'Bell curve', phân phối của nhiều hiện tượng tự nhiên

Câu A — Bernoulli Distribution: $P(X=1) = p$, $P(X=0) = 1-p$

Email spam: $p=0.3$. Tính xác suất các kịch bản:

P(email là spam) = 0.3

P(email không là spam) = 0.7

P(trong 5 email, đúng 2 cái là spam) = $C(5,2) \times 0.3^2 \times 0.7^3 =$ 0.3087



Các xác suất Bernoulli (điền số):

Câu B — Normal Distribution: điểm thi ML $\sim N(\mu=70, \sigma=12)$:

```
from scipy.stats import norm
```

P(điểm > 80) = $\text{norm.sf}(80, 70, 12) =$ 0.2023

P(điểm < 50) = $\text{norm.cdf}(50, 70, 12) =$ 0.0478

Ngưỡng top 10% = $\text{norm.ppf}(0.9, 70, 12) =$ 85.38



P(điểm > 80) $\approx$



P(điểm < 50) $\approx$

 Ngưỡng điểm top 10% $\approx$

Câu C — Quy tắc 68-95-99.7:

Với $N(70, 12)$: khoảng $1\sigma = [58, 82]$, khoảng $2\sigma = [46, 94]$

 % sinh viên nằm trong khoảng $[58, 82]$ theo lý thuyết: 68%

 % sinh viên nằm trong khoảng $[46, 94]$ theo lý thuyết: 95%

Bài 3.5 — Kỳ Vọng & Phương Sai

$$E[X] = \sum x_i \cdot P(x_i) \quad | \quad \text{Var}(X) = E[(X - E[X])^2] = \sum P(x_i)(x_i - \mu)^2$$

Bài toán: Chọn model nào để deploy?

Model A	Accuracy	Xác suất
	0.80	0.20
	0.82	0.30
	0.84	0.25
	0.83	0.15
	0.81	0.10

Model B	Accuracy	Xác suất
	0.70	0.25
	0.78	0.35

	0.88	0.25
	0.92	0.15

Câu A — Tính E[A] và E[B] ($E[X] = \sum x_i \times P(x_i)$):

$$\begin{aligned}
 E_A &= 0.80 \times 0.20 + 0.82 \times 0.30 + 0.84 \times 0.25 + 0.83 \times 0.15 + 0.81 \times 0.10 \\
 &= \underline{\quad} + \underline{\quad} + \underline{\quad} + \underline{\quad} + \underline{\quad} = \underline{\quad} \\
 E_B &= 0.70 \times 0.25 + 0.78 \times 0.35 + 0.88 \times 0.25 + 0.92 \times 0.15 \\
 &= \underline{\quad} + \underline{\quad} + \underline{\quad} + \underline{\quad} = \underline{\quad}
 \end{aligned}$$

 **E[A] = 0.8215**

 **E[B] = 0.806**

Câu B — Tính Var(A) và Var(B): $\text{Var}(X) = \sum P(x_i)(x_i - E[X])^2$:

 **Var(A) =**

Var(A) = 0.000203

 **Var(B) =**

Var(B) = 0.006364

Câu C — Quyết định deploy model nào và giải thích:

 **Model được chọn và lý do (xét cả kỳ vọng và variance):**

Chọn Model A.

=> Lý do: Model A vừa có độ chính xác trung bình cao hơn, vừa mang lại mức độ tin cậy và ổn định tốt hơn khi suy luận trên thực tế do mức độ phân tán rủi ro thấp (Variance nhỏ hơn hẳn Model B).

Bài 3.6 — Covariance & Correlation

$$\text{Cov}(X, Y) = E[(X - \mu_X)(Y - \mu_Y)] \quad | \quad \rho = \text{Cov}(X, Y) / (\sigma_X \times \sigma_Y) \in [-1, 1]$$

Dữ liệu 5 sinh viên:

Sinh viên	GRE	CGPA	Chance_Admit
A	310	7.8	0.65
B	325	8.5	0.80
C	295	7.2	0.52
D	335	9.1	0.90
E	305	8.0	0.68

Câu A — Tính Cov(GRE, Admit) thủ công:

```
GRE = [310, 325, 295, 335, 305] → mean_GRE = ____
Admit = [0.65, 0.80, 0.52, 0.90, 0.68] → mean_Admit = ____
# Cov = (1/5) × Σ (GRE_i - mean_GRE) (Admit_i - mean_Admit)
```

Sinh viên	GRE-mean	Admit-mean	(GRE-mean)(Admit-mean)
A	310-314=-4	0.65-0.71=-0.06	(-4)×(-0.06)=0.24
B	0.80 - 0.71 = 0.09	325 - 314 = 11	11 x 0.09 = 0.99
C	0.52 - 0.71 = -0.19	295 - 314 = -19	(-19) x (-0.19) = 3.61
D	0.90 - 0.71 = 0.19	335 - 314 = 21	21 x 0.19 = 3.99
E	0.68 - 0.71 = -0.03	305 - 314 = -9	(-9) x (-0.03) = 0.27
Tổng			9.10

 Cov(GRE, Admit) = Tổng/5 = 1.82

Câu B — Tính Correlation ρ :

```
std_GRE = np.std([310, 325, 295, 335, 305]) = ____
std_Admit = np.std([0.65, 0.80, 0.52, 0.90, 0.68]) = ____
ρ = Cov / (std_GRE × std_Admit) = ____ / (____ × ____) = ____
```

 Correlation ρ (GRE, Admit) = 0.9787

💬 $p > 0.5$ có nghĩa gì? GRE có nên là feature quan trọng trong model không?



Giải thích:

Thể hiện sự tương quan thuận tương đối mạnh. Khi GRE tăng, tỷ lệ nhận cũng có xu hướng tăng rõ rệt. Do đó, GRE rất nên là một feature quan trọng để đưa vào mô hình.

BÀI TỔNG HỢP — Logistic Regression Từ Đầu

Mục tiêu: Kết nối cả 3 chương để tự xây Logistic Regression. KHÔNG dùng sklearn. Chỉ dùng NumPy + toán thuần túy.

Phần A — Nền tảng lý thuyết (điền vào chỗ trống)

1. Sigmoid function — chuyển output thành xác suất [0,1]:

$$\sigma(z) = 1 / (1 + e^{(-z)})$$
$$\sigma(0) = \underline{0.5} \quad \sigma(+\infty) \approx \underline{1} \quad \sigma(-\infty) \approx \underline{0}$$

 Điền 3 giá trị trên:

2. Binary Cross-Entropy Loss — đo sai số xác suất:

$$L = -(1/n) \sum [y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i)]$$

Khi $\hat{y} = y = 1$: $L_i = \underline{0}$

Khi $\hat{y} = 0.5, y = 1$: $L_i = \underline{0.693}$ (sai nhiều $\rightarrow$ loss cao)

 Điền 2 giá trị L_i :

3. Gradient của BCE loss theo w (kết quả sau chain rule):

$$\partial L / \partial w = (1/n) X.T @ (\hat{y} - y)$$
$$\partial L / \partial b = (1/n) \sum (\hat{y} - y)$$

$(\hat{y} - y)$ là vector 'error'. Shape của $X.T @ \text{error} = \underline{\hspace{2cm}}$

 Shape của gradient $\partial L / \partial w$ (bằng shape của w):

$$\partial L / \partial w = (1/n) X.T @ (y_hat - y)$$
$$\partial L / \partial b = (1/n) \sum (y_hat - y)$$

$(\hat{y} - y)$ là vector 'error'. Shape của $X.T @ \text{error} = (n_features,)$

Phần B — Implement từng hàm (viết code)

Bước 1 — Sigmoid function:

```
def sigmoid(z):  
    return ____ # điền công thức
```

Code sigmoid:

```
def sigmoid(z):  
    return 1 / (1 + np.exp(-z))
```

Bước 2 — BCE Loss:

```
def bce_loss(y_true, y_pred):  
    y_pred = np.clip(y_pred, 1e-10, 1-1e-10)  
    return ____ # công thức BCE
```

Code bce_loss:

```
def bce_loss(y_true, y_pred):  
    y_pred = np.clip(y_pred, 1e-10, 1-1e-10)  
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
```

Bước 3 — Training loop (1 epoch):

```
# Forward pass:  
z = ____ # X @ w + b  
y_hat = ____ # sigmoid(z)  
loss = ____ # bce_loss(y, y_hat)  
  
# Backward pass (gradient):  
error = ____ # y_hat - y  
dw = ____ # X.T @ error / n  
db = ____ # mean(error)  
  
# Update:  
w = ____ # w - alpha * dw  
b = ____ # b - alpha * db
```

Điền 8 chỗ trống (có thể viết bên cạnh code):

```
z = X @ w + b  
y_hat = sigmoid(z)  
loss = bce_loss(y, y_hat)  
error = y_hat - y  
dw = X.T @ error / n  
db = np.mean(error)  
w = w - alpha * dw  
b = b - alpha * db
```

Phần C — Phân tích kết quả

Sau 200 epochs, quan sát loss curve và viết nhận xét:

Loss giảm từ ____ (epoch 1) xuống ____ (epoch 200)?

 **Accuracy trên training set $\approx$**

💬 Nếu loss không giảm, nguyên nhân có thể là gì? (3 nguyên nhân phổ biến)

 **Nguyên nhân 1:**

Learning rate (α) quá nhỏ (hội tụ quá chậm) hoặc quá lớn (dao động vượt qua điểm cực tiểu).

 **Nguyên nhân 2:**

Dữ liệu (Features X) chưa được chuẩn hóa (Scaling/Normalization), khiến không gian gradient bị lệch.

 **Nguyên nhân 3:**

Tính sai công thức backward pass (đạo hàm dw hoặc db sai) dẫn đến đi chệch hướng hội tụ.

Câu Hỏi Ôn Tập Cuối Phần

Trả lời các câu hỏi dưới đây để kiểm tra mức độ nắm vững kiến thức. Không nhìn slide — cố gắng trả lời từ trí nhớ.

Phần 1 — Linear Algebra

Q1. Dot product $w \cdot x$ có ý nghĩa gì trong một linear model? Cho ví dụ cụ thể.

 Trả lời:

Ý nghĩa: Tính tổng có trọng số (weighted sum) của tất cả các đặc trưng đầu vào.
Ví dụ: Tính điểm đánh giá nhà $Y = (\text{Diện tích} \times 0.6) + (\text{Số phòng} \times 0.4)$.

Q2. Tại sao matrix multiplication $X@W$ hiệu quả hơn vòng for khi x có 1 triệu dòng?

 Trả lời:

Cơ chế Vectorization: Phép nhân ma trận được xử lý song song ở cấp độ phần cứng (CPU/GPU) thông qua các thư viện toán học tối ưu (BLAS, LAPACK) thay vì lặp tuần tự từng dòng.

Q3. Transpose X^T xuất hiện trong công thức nào của gradient descent? Tại sao cần T ?

 Trả lời:

Công thức: Tính gradient trọng số $dw = (1/n) X^T$.

Lý do: Đảo chiều ma trận để đồng bộ kích thước (shape matching). Chuyển X từ (n, d) sang X^T (d, n) để nhân với vector error $(n, 1)$, trả về gradient có chiều $(d, 1)$ khớp với trọng số w .

Phần 2 — Calculus

Q4. Gradient trở về hướng nào? Tại sao Gradient Descent đi ngược lại?

 Trả lời:

Hướng Gradient: Trở về hướng hàm số tăng dốc nhất (steepest ascent).

Lý do đi ngược: Mục tiêu là cực tiểu hóa (minimize) hàm Loss, nên phải bước ngược hướng đạo hàm để đi xuống đáy.

Q5. Giải thích chain rule trong bối cảnh backpropagation — 3 hàm lồng nhau.

 Trả lời:

Phân rã sai số từ output ngược về input để đo lường mức độ đóng góp của từng trọng số w vào sai số tổng (Loss).

Q6. Learning rate $\alpha = 10$ gây ra vấn đề gì? $\alpha = 0.000001$ gây ra vấn đề gì?

 α quá lớn:

$\alpha = 10$ (Quá lớn): Bước nhảy vượt lố điểm cực tiểu (overshooting), khiến hàm loss dao động hoặc phân kỳ.

 α quá nhỏ:

$\alpha = 0.000001$ (Quá nhỏ): Hội tụ cực kỳ chậm, tốn tài nguyên và dễ mắc kẹt ở cực tiểu địa phương (local minima).

Phần 3 — Probability & Statistics

Q7. Trong Bayes Theorem, $P(A)$ gọi là gì? $P(A|B)$ gọi là gì? Ý nghĩa thực tế?

 Trả lời:

$P(A)$ (Prior): Xác suất tiên nghiệm - Niềm tin/xác suất ban đầu trước khi có dữ liệu quan sát.

$P(A|B)$ (Posterior): Xác suất hậu nghiệm - Niềm tin được cập nhật sau khi đã thu thập bằng chứng B.

Q8. Correlation cao giữa feature X và Y có nghĩa X gây ra Y không? Giải thích.

 Trả lời:

Không. Tương quan không đồng nghĩa với nhân quả (Correlation does not imply causation). Hai biến có thể cùng thay đổi do tác động của một biến ẩn thứ ba (confounding variable).

Q9. Khi nào nên dùng L1 (Lasso) thay vì L2 (Ridge) regularization?

 Trả lời:

Ứng dụng L1: Khi cần chọn lọc đặc trưng (Feature Selection). L1 ép các trọng số không quan trọng về chính xác bằng 0, tạo ra mô hình thưa (sparse model).

Q10. Logistic Regression dùng Sigmoid thay vì output thẳng. Tại sao?



Trả lời:

Hàm Sigmoid ép (squash) đầu ra tuyến tính từ khoảng $(-\infty; +\infty)$ về khoảng $[0, 1]$, giúp chuẩn hóa kết quả thành dạng xác suất (probability) để phục vụ bài toán phân loại.

—— Hoàn thành bài tập = Nền tảng vững để học ML Algorithms! ——

Machine Learning in Data Sciences | Instructor: Lai Hien Phuong | PhuongLH17@fe.edu.vn